Laboration 2: Cellular Automata

Group Supermodels
Vootele Mets
Juan Rodriguez
Marco Malosti
Björk Lucas
Sofia Fernandes
David Weingut

May 5, 2026

UPPSALA
UNIVERSITET

Contents

1 Problem statement **2**

2 Game of Life **3**
- 2.1 Numerical methods for Game of Life . . . 3
  - 2.1.1 Computational domain . . . 3
  - 2.1.2 Time evolution . . . 3
  - 2.1.3 Initial conditions . . . 3
  - 2.1.4 Logical gates . . . 3
- 2.2 Simulation Results . . . 4
- 2.3 NOT-gate . . . 4
- 2.4 Initial conditions . . . 4
- 2.5 Transient state . . . 4
- 2.6 AND-gate . . . 5
- 2.7 Initial conditions . . . 5
- 2.8 Transient state . . . 6
- 2.9 OR-gate . . . 7
- 2.10 Initial conditions . . . 7
- 2.11 Transient state . . . 8

3 Neuron Cellular Automaton **9**
- 3.1 Neuron Automaton Theory . . . 9
- 3.2 Neuron Cellular Automaton . . . 9
- 3.3 Neuron Automaton Results . . . 9

4 Greenberg-Hastings Cellular Automaton **12**
- 4.1 GHCA theory . . . 12
- 4.2 GHCA Method . . . 12
- 4.3 GHCA Results . . . 12

5 Interpretation **13**

6 Conclusion **14**

7 Use of LLM tools **14**
- 7.1 Game of Life . . . 14
- 7.2 Neuron Cellular Automaton . . . 14

1. Problem statement

In this lab we will be investigating classical examples of cellular automata. Cellular automata are ...

The goal of the lab is to ...

All of these tasks will give us an overview of the characteristics in cellular automata ...

2. Game of Life

2.1. Numerical methods for Game of Life

2.1.1. Computational domain

The computational domain is represented as a two-dimensional rectangular lattice $X \in \{0, 1\}^{n \times n}$, where each entry corresponds to one cell of the automaton.

2.1.2. Time evolution

The time evolution is implemented as a composition of a linear convolution operation and a pointwise non-linear update. The discrete convolution is computed using `scipy.ndimage.convolve`. A fixed 3×3 kernel is used to compute the Moore neighbourhood sum $N^{(t)} = K * X^{(t)}$.. The central entry of the kernel is zero, so the cell itself is excluded from the neighbour count. The update function allows both zero-padding boundaries, set by the parameter `mode="constant"`.

The Game of Life update rule is then applied by vectorized logical operations:

$$X^{(t+1)} = \mathbb{1}\left(N^{(t)} = 3\right) \vee \left[X^{(t)} \wedge \mathbb{1}\left(N^{(t)} = 2\right)\right]. \tag{2.1}$$

2.1.3. Initial conditions

Initial conditions are generated programmatically in `GoLogic.py`. The basic building blocks are the Gosper glider gun and the eater, both stored in Run Length Encoding (RLE). A custom parser converts each RLE string into a binary array using `RLE.py` and `add_pattern`. Before adding a pattern one can transform it appropriately with the `transform_pattern` function in `pattern_processing.py`. handle insertion and transformation before simulation.

In addition, the function `load_live_cells(filename, n)` loads external initial configurations from text files containing one live cell per line in the format `(x,y)`. The file is converted into an $n \times n$ binary array by setting `grid[y,x] = 1`. This supports the coordinate-list representation used for the initial gate configurations.

2.1.4. Logical gates

The logical gates `AND`, `OR`, and `NOT` are implemented by spatial arrangements of glider guns and eaters. Signals are encoded as glider streams emitted by Gosper guns. Boolean inputs are represented through the initial configuration: an input value of 1 leaves the corresponding signal unobstructed, whereas an input value of 0 places an eater on the signal path and suppresses propagation.

For the `AND` and `OR` gates, all four input combinations were simulated,

$$(A, B) \in \{(0, 0), (0, 1), (1, 0), (1, 1)\},$$

while the `NOT` gate was tested for both possible input values,

$$A \in \{0, 1\}.$$

The output was read from a designated output lane: the presence of a propagating glider stream was interpreted as logical 1, and its absence as logical 0.

2.2. Simulation Results

2.3. NOT-gate

2.4. Initial conditions

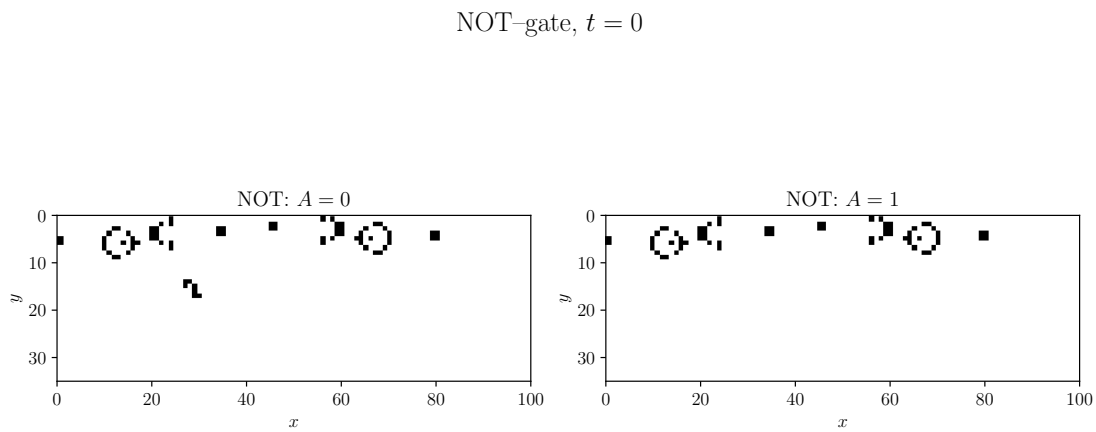

Figure 1. Two Gosper glider guns: the left gun provides a constant input stream, while the input channel injects gliders traveling in the south-east direction. No input is encoded by inserting an eater in the SE input stream. The output is read as gliders propagating in the south-west direction.

2.5. Transient state

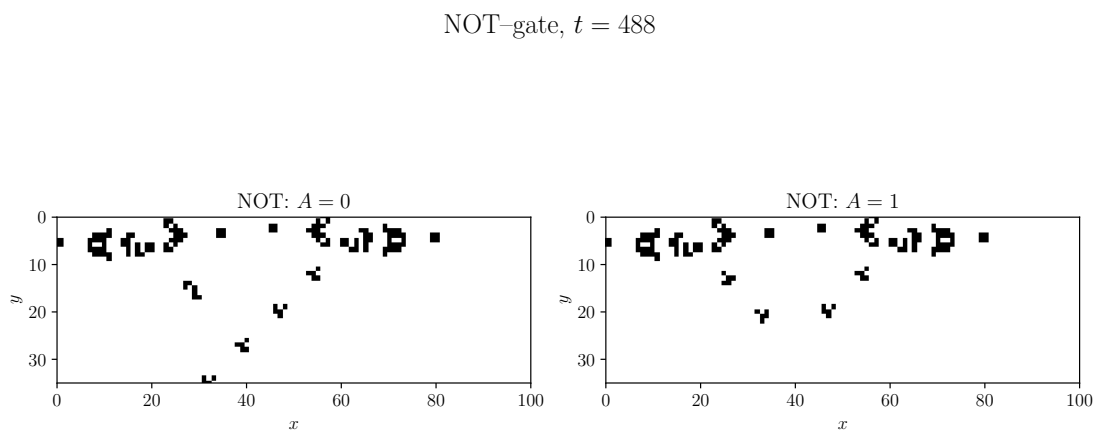

Figure 2. One input channel, A. The input glider moves diagonally from the top to the lower-right part of the grid. The output is observed after the interaction region, in the outgoing diagonal channel. The construction is arranged so that when $A = 0$ an output glider survives, while when $A = 1$ the input glider destroys or blocks the outgoing signal.

2.6. AND-gate

2.7. Initial conditions

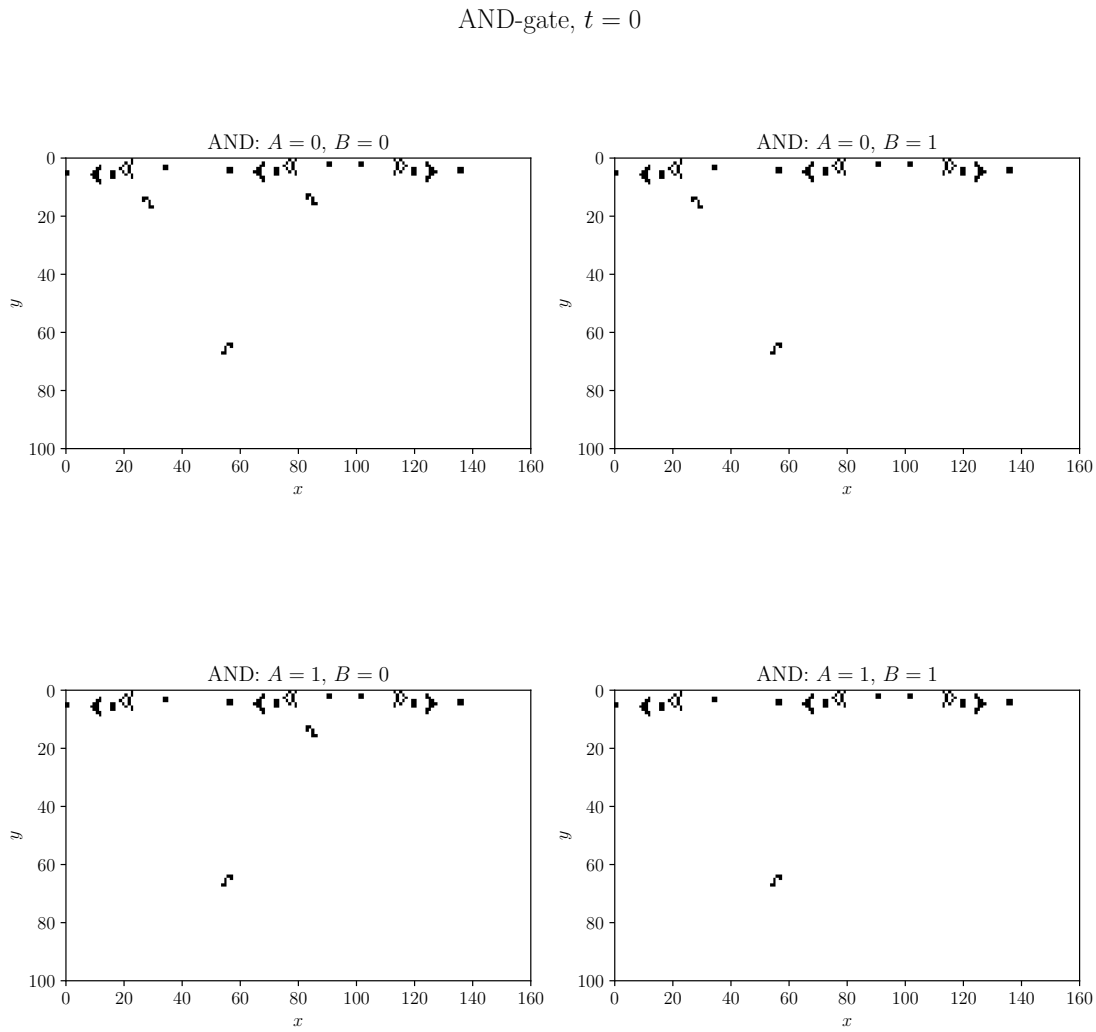

Figure 3. Three Gosper glider guns: the two leftmost generate south-east-directed input streams. An eater suppresses the south-west output unless both inputs are present. In the absence of inputs, additional eaters are placed along the collision paths to block spurious gliders. The output is read as a south-east-directed stream.

44

2.8. Transient state

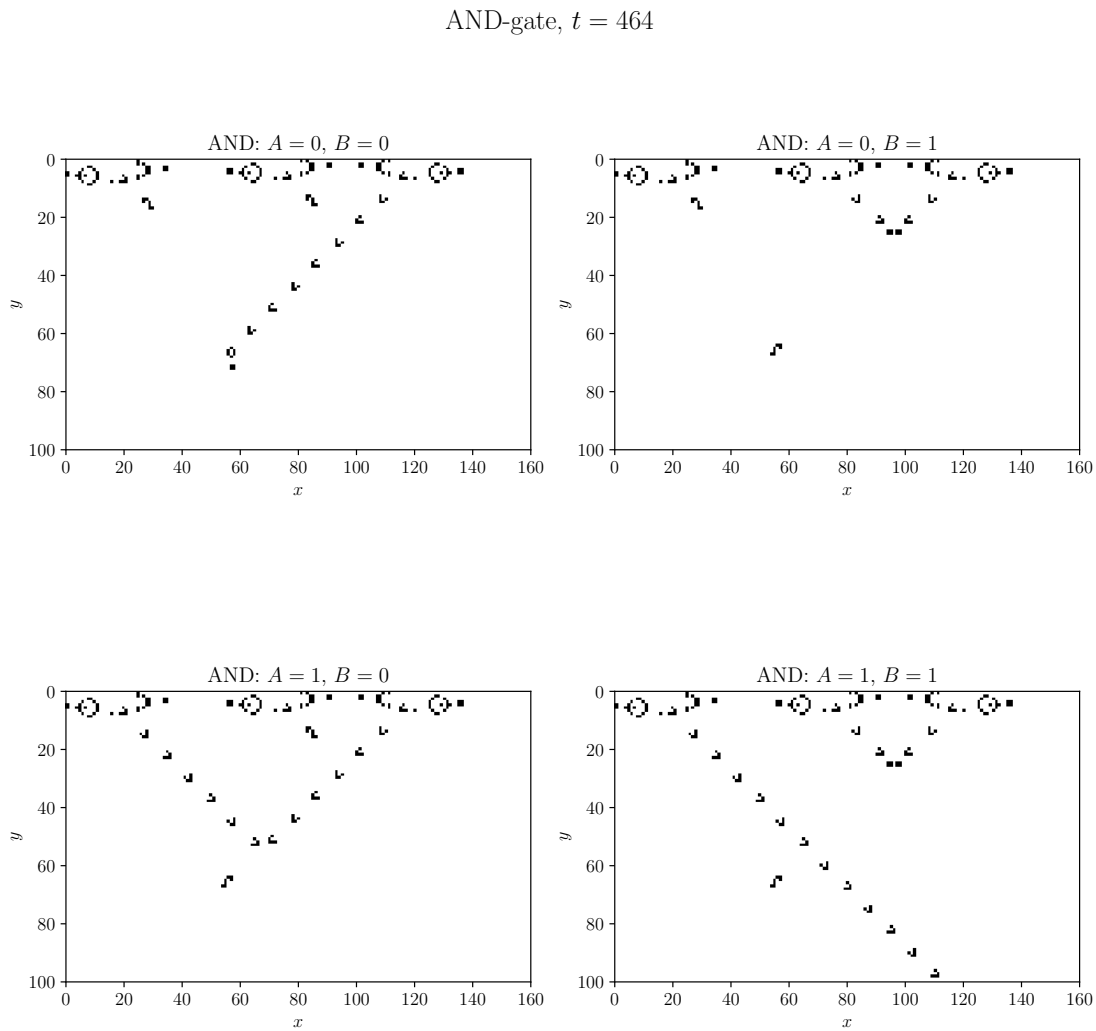

Figure 4. Two input channels are used, A and B. The input gliders move diagonally from the top of the grid towards the lower-right part of the grid, where they interact with the gate structure. The output is observed in the outgoing diagonal channel after this interaction. A glider appears in the output channel only when both input gliders are present, so the output is 1 only for $A = 1$ and $B = 1$.

2.9. OR-gate

2.10. Initial conditions

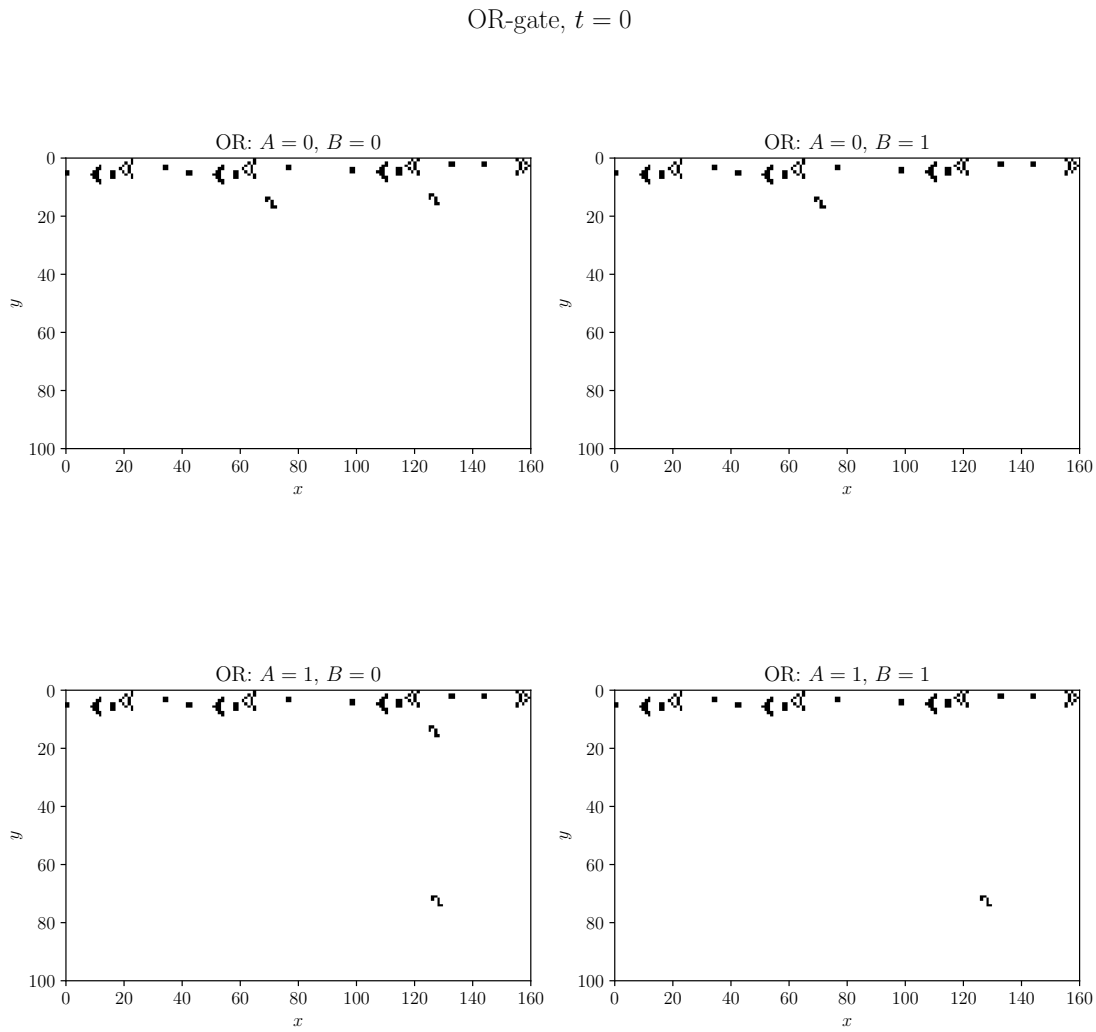

Figure 5. Four Gosper glider guns: the second and third generate south-east-directed input streams. Logical zeros are implemented by placing eaters along the input collision paths. The output is read as a south-east-directed stream.

47

2.11. Transient state

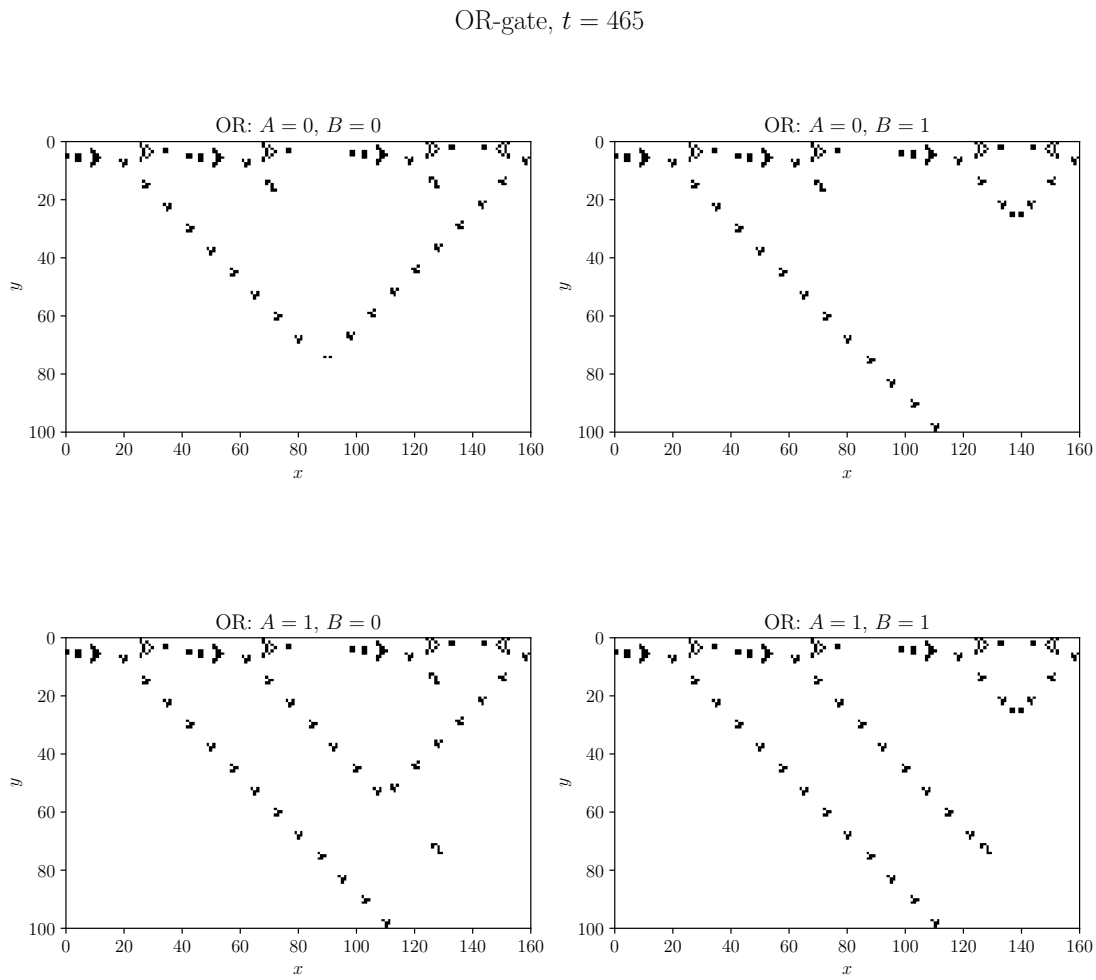

Figure 6. Input gliders also move diagonally from the top of the grid towards the lower-right part of the grid. The output is shown in the outgoing diagonal channel after the region of collision. In this case, the output channel contains a glider when, at least, one of the two input gliders is present.

3. Neuron Cellular Automaton

3.1. Neuron Automaton Theory

FOR THEORY OF GHCA...

3.2. Neuron Cellular Automaton

The script for the implementation of the neuron cellular automaton is `neuron.py`. The code is based on `gameOfLifeInteractiveToric.py`, which was available on the Studium page of the course.

Now, we will present how the code works. At first, the number of rows and columns of the grid as well the speed of the simulation are defined. Additionally, the rules text is added. This text is displayed when clicking the ”rules”-button while running the script. Next, we have implemented the new function `count_neighbors_by_state` that can calculate the number of neighbours in a certain state. This function is necessary as we will be working with three different states instead of two. The function takes the grid and the state that we want to determine the number of neighbours for as arguments. The neighbours for each gridpoint are found using `numpy.roll` to include the periodic boundary conditions. As the next step we have updated the `lifeStep` function. Now, we first create the grids of neighbours in states 1 and 2 and apply the rules afterwards. State 0 represents the resting neurons, state 1 the ready neurons and state 2 the firing neurons. The function then returns the updated grid.

In the constructor of the `GameOfLifeApp` class the visualisation is defined. This includes the buttons that are displayed and their actions. The time step for the continuous run is defined as well.

The following parameters are fixed in the code: the number of rows and columns is set to 30 each. For the random population of the grid, the probability for state 0 is 0.7, for state 1 the probability is 0.2 and 0.1 for state 2. Initially, all cells are in state 0 (resting). Clicking once sets them to state 1 and clicking another time to state 2.

3.3. Neuron Automaton Results

Using random initial conditions in the neuron automation usually led to either the whole system eventually dying out and being inert in the ”ready” condition or to infinitely forward moving gliders appearing. By experimenting several initial conditions were constructed that would lead to interesting developments of the system such as a periodic oscillator or a pattern that infinitely generates more moving patterns. Some interesting initial conditions are pictured below.

First a number of infinitely moving gliders were identified. On figure 7 are brought three examples. Those kinds of patterns move infinitely in the initial direction unless they collide with another pattern.

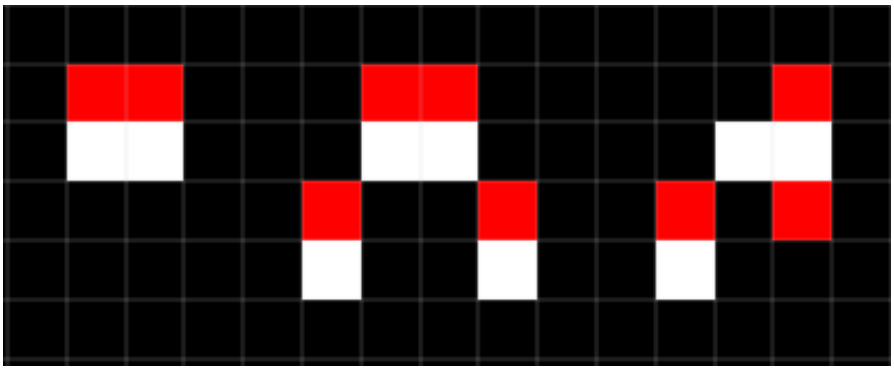

Figure 7. Three examples of infinitely moving patterns. Two patterns on the left move in the upward direction and the one on the right moves diagonally to right-left.

By having two simple gliders collide with eachother using initial conditions shown in figure 8 it is possible to create two new gliders that move at a right angle to the initial motion giving the impression of two bouncing gliders.

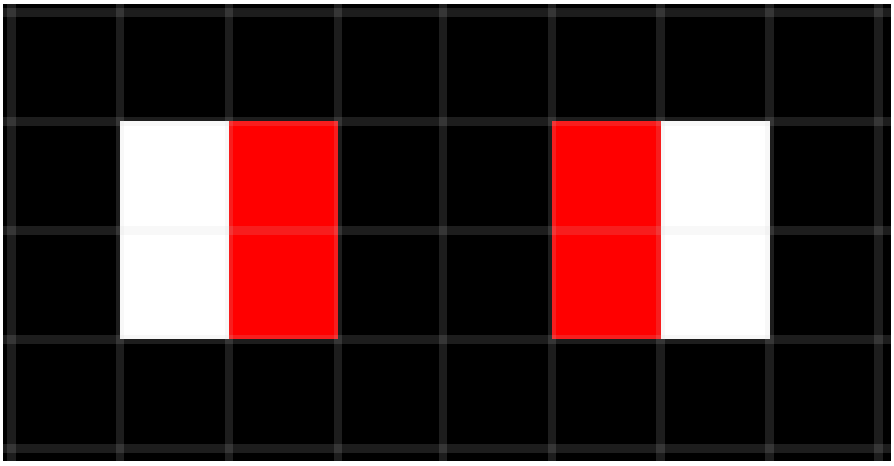

Figure 8. Initial condition that leads to two gliders bouncing in a 90 degree angle

It is also possible to set up a pattern that stais in the same location and oscillates infinitely. An oscillator with three distinct steps was identified that is pictured in figure 9

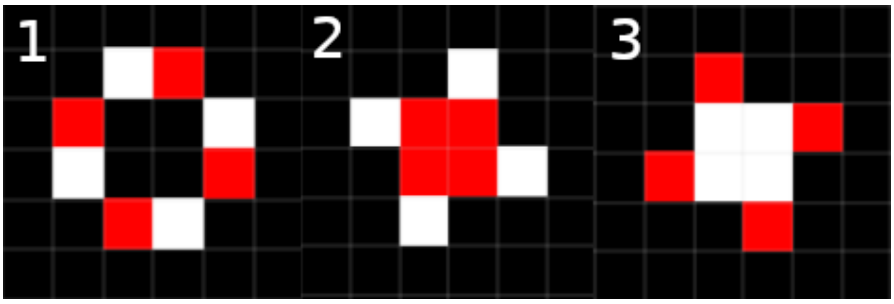

Figure 9. A three step oscillating pattern

On figure 10 is pictured a pattern that moves forward and periodically creates more moving patterns.

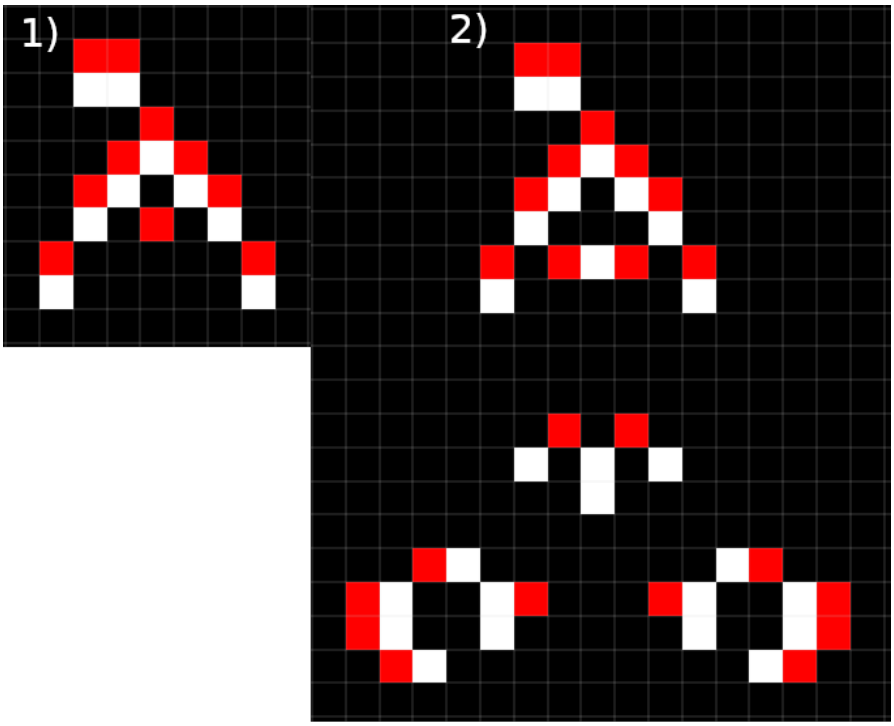

Figure 10. A pattern that while moving forward creates other moving patterns. 1) initial condition 2) system after some steps. the bottom two shapes are gliders moving to the left and right.

4. Greenberg-Hastings Cellular Automaton

4.1. GHCA theory

GHCA THEORY HERE

4.2. GHCA Method

FOR IMPLEMENTATION OF GHCA...

4.3. GHCA Results

FOR RESULTS OF GHCA...

5. Interpretation

INTERPRETATION GOES HERE.

6. Conclusion

In Section (2.2) we presented snapshots illustrating the realization of logic gates within a computer simulation of the Game of Life. From this, one may conclude that a Boolean basis is effectively implemented within the system. Once the principles for composing these gates into larger circuits are understood, increasingly complex computational structures can be constructed.

The existence of logic gates in the Game of Life implies that they form a functionally complete basis for Boolean logic. Consequently, any Boolean function can be expressed as a finite composition of such gates. This property is fundamental for the construction of general computational devices. In particular, it is possible to design patterns that behave as a finite-state machine coupled to two counters. Such a system is computationally equivalent to a universal Turing machine

It follows that the Game of Life possesses the same theoretical computational power as any computer with unbounded memory and no time constraints; that is, it is Turing complete [1]. This demonstrates that complex dynamics can emerge solely from prescribed initial conditions and a simple update rule, forming a discrete dynamical system.

7. Use of LLM tools

7.1. Game of Life

RLE prompt: Create an rle loader to import standard Game of Life patterns directly. The output was a complete functional script `RLE.py` that parses strings like

```
24bo11b$22bobo11b$12b2o6b2...
```

, which encode binary patterns stored in a Wiki page.

7.2. Neuron Cellular Automaton

The code in `neuron.py` is based on `gameOfLifeInteractiveToric.py` from the lecture. We used ChatGPT to modify the code. The prompt was ”I would like to change the rules of the game. I need to have three different states for each cell now and new rules, how can I implement this?” and additionally added the original code. This returned good instructions on where the code needs modification. Additionally, we looked up the documentation of the functions used by ChatGPT and made some modifications to its suggestions.

Files submitted

The following files have been submitted with this report:

1. `GameOfLife.py`
2. `GameOfLifeDebug.py`
3. `GoLogic.py`
4. `pattern_processing.py`
5. `RLE.py`
6. `NOT.txt`
7. `AND.txt`
8. `OR.txt`

References

[1] Wikipedia. Turing completeness.